gatectr.io · Developer Documentation v1.0 · Confidential
© 2025 GateCtr — All rights reserved

01 Prérequis & Installation des outils

Tout ce dont vous avez besoin avant de coder

Cette section liste chaque outil à installer, la version recommandée et la commande de vérification. Suivez l'ordre indiqué pour garantir une configuration cohérente sur macOS, Linux ou WSL2 (Windows).

Outils système requis

Outil	Version min.	Installation	Vérification
Node.js	20 LTS	nvm install 20 / brew install node	`node -v`
npm / pnpm	npm 10 / pnpm 8	npm i -g pnpm	`pnpm -v`
Git	2.40+	brew install git / apt install git	`git --version`
Docker Desktop	24+	hub.docker.com/editions/community	`docker -v`
Docker Compose	v2.x	Inclus dans Docker Desktop	`docker compose version`
PostgreSQL	16 (via Docker)	Via docker-compose.yml	`docker exec -it pg psql`
Redis	7.x (via Docker)	Via docker-compose.yml	`docker exec -it redis redis-cli`
VS Code / Cursor	Latest	code.visualstudio.com	-
Prisma CLI	5.x	pnpm add -D prisma	`npx prisma -v`

Installation Node.js via nvm (recommandé)

```
# Installer nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

# Recharger le shell
source ~/.bashrc  # ou ~/.zshrc

# Installer Node 20 LTS
nvm install 20
nvm use 20
nvm alias default 20

# Vérifications
node -v    # v20.x.x
npm -v     # 10.x.x

# Installer pnpm (package manager recommandé)
npm install -g pnpm@latest
pnpm -v    # 8.x.x
```

Installation Docker

© 2025 GateCtr

```
# macOS
brew install --cask docker
# Lancer Docker Desktop puis vérifier :
docker -v && docker compose version

# Linux (Ubuntu / Debian)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
docker -v && docker compose version

# WSL2 (Windows)
# Installer Docker Desktop avec option WSL2 Backend
# https://docs.docker.com/desktop/windows/wsl/
```

Extensions VS Code / Cursor recommandées

Extension	ID	Usage
ESLint	`dbaeumer.vscode-eslint`	Linting TypeScript/JS
Prettier	`esbenp.prettier-vscode`	Formatage auto du code
Prisma	`Prisma.prisma`	Syntaxe .prisma, autocomplétion
Tailwind CSS IntelliSense	`bradlc.vscode-tailwindcss`	Classes Tailwind
Docker	`ms-azuretools.vscode-docker`	Gestion conteneurs dans IDE
GitLens	`eamodio.gitlens`	Git history, blame, branches
REST Client	`humao.rest-client`	Tester les API directement
Error Lens	`usernamehako.error-lens`	Erreurs inline dans l'éditeur
Thunder Client	`rangav.vscode-thunder-client`	Client HTTP intégré

■■ Cursor IDE (fork VS Code + IA) est recommandé pour ce projet. Toutes les extensions VS Code sont compatibles.

© 2025 GateCtr

02 Structure du Projet & Premier Git

Monorepo Next.js, architecture & initialisation

Créer le projet Next.js

```
# Créer le projet avec Next.js 14 + TypeScript + Tailwind + App Router
pnpm create next-app@latest gatectr \
  --typescript \
  --tailwind \
  --eslint \
  --app \
  --src-dir \
  --import-alias "@/*"

cd gatectr

# Installer les dépendances principales
pnpm add @clerk/nextjs @prisma/client zod axios
pnpm add @tanstack/react-query zustand recharts lucide-react
pnpm add bullmq ioredis @upstash/redis
pnpm add next-themes clsx tailwind-merge
pnpm add @radix-ui/react-dialog @radix-ui/react-dropdown-menu

# Dépendances de développement
pnpm add -D prisma @types/node tsx vitest
pnpm add -D @testing-library/react @testing-library/jest-dom
pnpm add -D playwright @playwright/test
pnpm add -D husky lint-staged prettier eslint-config-prettier
```

Structure des dossiers

```
gatectr/
■■■ src/
■   ■■■ app/                        # Next.js App Router
■   ■   ■■■ (auth)/                 # Route group : login, signup
■   ■   ■   ■■■ sign-in/
■   ■   ■   ■■■ sign-up/
■   ■   ■■■ (dashboard)/            # Route group : app authentifiée
■   ■   ■   ■■■ layout.tsx          # Layout dashboard avec sidebar
■   ■   ■   ■■■ page.tsx            # Overview / home
■   ■   ■   ■■■ analytics/          # Token usage analytics
■   ■   ■   ■■■ projects/           # Gestion des projets
■   ■   ■   ■■■ budget/             # Budget firewall config
■   ■   ■   ■■■ webhooks/           # Webhook manager
■   ■   ■   ■■■ settings/           # Paramètres compte + clés API
■   ■   ■■■ api/                    # API Routes Next.js
■   ■   ■   ■■■ v1/
■   ■   ■   ■   ■■■ complete/       # POST /api/v1/complete
■   ■   ■   ■   ■■■ chat/           # POST /api/v1/chat
■   ■   ■   ■   ■■■ usage/          # GET  /api/v1/usage
■   ■   ■   ■   ■■■ budget/         # GET/POST /api/v1/budget
■   ■   ■   ■   ■■■ webhooks/       # CRUD /api/v1/webhooks
■   ■   ■   ■   ■■■ logs/           # GET  /api/v1/logs
```


© 2025 GateCtr

```
■   ■   ■   ■   ■■■ models/         # GET  /api/v1/models
■   ■   ■   ■■■ webhooks/
■   ■   ■       ■■■ clerk/          # Clerk webhook (user sync)
■   ■   ■■■ globals.css
■   ■   ■■■ layout.tsx              # Root layout
■   ■■■ components/                 # Composants React réutilisables
■   ■   ■■■ ui/                     # shadcn/ui components
■   ■   ■■■ dashboard/              # Composants dashboard
■   ■   ■■■ charts/                 # Charts analytics
■   ■   ■■■ layout/                 # Header, Sidebar, Footer
■   ■■■ lib/                        # Utilitaires et services
■   ■   ■■■ prisma.ts               # Prisma client singleton
■   ■   ■■■ redis.ts                # Redis client
■   ■   ■■■ queue.ts                # BullMQ queue definitions
■   ■   ■■■ optimizer.ts            # Context Optimizer logic
■   ■   ■■■ router.ts               # Model Router logic
■   ■   ■■■ firewall.ts             # Budget Firewall
■   ■   ■■■ webhooks.ts             # Webhook dispatcher
■   ■   ■■■ llm/                    # LLM provider adapters
■   ■       ■■■ openai.ts
■   ■       ■■■ anthropic.ts
■   ■       ■■■ mistral.ts
■   ■       ■■■ index.ts            # Unified LLM interface
■   ■■■ hooks/                      # React custom hooks
■   ■■■ types/                      # TypeScript types/interfaces
■   ■■■ middleware.ts               # Clerk auth middleware
■■■ prisma/
■   ■■■ schema.prisma               # Schéma base de données
■   ■■■ migrations/                 # Migrations générées
■■■ workers/                        # BullMQ workers (séparés)
■   ■■■ webhook.worker.ts
■   ■■■ analytics.worker.ts
■■■ tests/
■   ■■■ unit/                       # Tests unitaires (Vitest)
■   ■■■ integration/                # Tests d'intégration
■   ■■■ e2e/                        # Tests E2E (Playwright)
■■■ docker/
■   ■■■ Dockerfile                  # Production
■   ■■■ Dockerfile.dev              # Développement
■   ■■■ nginx.conf                  # Config reverse proxy
■■■ .github/
■   ■■■ workflows/                  # GitHub Actions CI/CD
■       ■■■ ci.yml
■       ■■■ deploy.yml
■■■ docker-compose.yml              # Dev stack complète
■■■ docker-compose.prod.yml         # Production stack
■■■ .env.example                    # Template variables env
■■■ .env.local                      # Variables locales (gitignored)
■■■ package.json
```

Premier Git — Initialisation & Convention

```
# Initialiser Git (si pas encore fait par create-next-app)
git init
git branch -M main
```


© 2025 GateCtr

```
# Configurer l'identité locale (si besoin)
git config user.name "Votre Nom"
git config user.email "vous@email.com"

# Créer le .gitignore (Next.js le crée auto, vérifier)
cat >> .gitignore << 'EOF'
.env.local
.env.*.local
.DS_Store
*.log
/workers/dist
EOF

# Premier commit
git add .
git commit -m "chore: initial Next.js project setup

- Next.js 14 with App Router + TypeScript
- Tailwind CSS + shadcn/ui
- ESLint + Prettier configuration
- Project structure and folder organization"

# Connecter au remote GitHub
git remote add origin https://github.com/votre-org/gatectr.git
git push -u origin main
```

Convention de commits (Conventional Commits)

Type	Usage	Exemple
`feat`	Nouvelle fonctionnalité	`feat(api): add /v1/complete endpoint`
`fix`	Correction de bug	`fix(optimizer): handle empty context`
`chore`	Maintenance, tooling, config	`chore: add husky pre-commit hooks`
`docs`	Documentation	`docs: update README with Docker setup`
`test`	Ajout ou modif de tests	`test(budget): add firewall unit tests`
`refactor`	Refactoring sans changement fonc.	`refactor(router): extract scoring logic`
`perf`	Amélioration de performance	`perf(cache): add Redis semantic cache`
`ci`	CI/CD pipeline changes	`ci: add Playwright E2E to GitHub Actions`
`style`	Formatage, espacements	`style: run prettier on all files`

Branching Strategy (GitHub Flow)

```
# Branche principale
main          # Production stable — jamais de push direct

# Branche de développement
```


© 2025 GateCtr

```
develop       # Intégration continue des features

# Feature branches (depuis develop)
git checkout develop
git pull origin develop
git checkout -b feat/budget-firewall

# Hotfix (depuis main)
git checkout main
git checkout -b hotfix/fix-token-overflow

# Exemple de workflow complet
git checkout -b feat/context-optimizer
# ... développement ...
git add .
git commit -m "feat(optimizer): implement context compression"
git push origin feat/context-optimizer
# → Ouvrir une Pull Request vers develop
# → Code review
# → Merge avec squash
# → Delete branch
```


© 2025 GateCtr

03 Docker & Docker Compose

Environnements de développement et production containerisés

docker-compose.yml — Environnement de développement

```
# docker-compose.yml
version: '3.9'

services:
  # ■■ Application Next.js ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  app:
    build:
      context: .
      dockerfile: docker/Dockerfile.dev
    container_name: gatectr_app
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgresql://gatectr:secret@postgres:5432/gatectr_dev
      - REDIS_URL=redis://redis:6379
      - NEXT_PUBLIC_APP_URL=http://localhost:3000
    env_file:
      - .env.local
    volumes:
      - .:/app
      - /app/node_modules
      - /app/.next
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - gatectr_net
    restart: unless-stopped

  # ■■ PostgreSQL 16 ■■■■■■■■■■■■■■■■■■■
```

```
    timeout: 5s
    retries: 5
  networks:
    - gatectr_net

# ■■ Redis 7 ■■■
```

```
      - DATABASE_URL=postgresql://gatectr:secret@postgres:5432/gatectr_dev
      - REDIS_URL=redis://redis:6379
    env_file:
      - .env.local
    volumes:
      - .:/app
      - /app/node_modules
    depends_on:
      - redis
      - postgres
    networks:
      - gatectr_net

volumes:
  postgres_data:
  redis_data:

networks:
  gatectr_net:
    driver: bridge
```

Dockerfile.dev

```
# docker/Dockerfile.dev
FROM node:20-alpine AS base

RUN apk add --no-cache libc6-compat curl
WORKDIR /app

# Install pnpm
RUN npm install -g pnpm@latest

# Install dependencies
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile

COPY . .

# Generate Prisma client
RUN npx prisma generate

EXPOSE 3000

# Hot reload via Next.js dev server
CMD ["pnpm", "dev"]
```

Dockerfile — Production multi-stage

```
# docker/Dockerfile
FROM node:20-alpine AS base
RUN apk add --no-cache libc6-compat
RUN npm install -g pnpm@latest

# ■■ DEPS STAGE ■■■
```

```
FROM base AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile --prod

# ■■ BUILD STAGE ■■■
```

```
docker exec -it gatectr_redis redis-cli

# Rebuild après changement Dockerfile
docker compose up -d --build app

# Stopper et nettoyer (volumes inclus)
docker compose down -v

# Statut des conteneurs
docker compose ps
```

.env.local — Variables d'environnement

```
# .env.local (copier depuis .env.example et remplir)

# ■■ DATABASE ■■■
```

PHASE 1

Setup & Configuration

Auth Clerk · Prisma · Redis · Structure de base

1

04 Phase 1 — Setup & Configuration

Clerk Auth, Prisma Schema, Redis, Middleware

1.1 — Configurer Clerk Auth

```
# src/middleware.ts
import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';

const isPublicRoute = createRouteMatcher([
  '/',
  '/sign-in(.*)',
  '/sign-up(.*)',
  '/api/v1/(.*)',   // Les appels API SDK sont publics (auth par apiKey)
]);

export default clerkMiddleware((auth, req) => {
  if (!isPublicRoute(req)) {
    auth().protect();
  }
});

export const config = {
  matcher: ['/((?!.*\..*|_next).*)', '/', '/(api|trpc)(.*)'],
};
```

```
// src/app/layout.tsx
import { ClerkProvider } from '@clerk/nextjs';
import { Inter } from 'next/font/google';
import './globals.css';

export default function RootLayout({ children }) {
  return (
    <ClerkProvider>
      <html lang="fr">
        <body>{children}</body>
      </html>
    </ClerkProvider>
  );
}
```

1.2 — Schéma Prisma (PostgreSQL)

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}
```


© 2025 GateCtr

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ■■ USER (synced from Clerk) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
model User {
  id          String    @id @default(cuid())
  clerkId     String    @unique
  email       String    @unique
  name        String?
  plan        Plan      @default(FREE)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  apiKeys     ApiKey[]
  projects    Project[]
  webhooks    Webhook[]
  usageLogs   UsageLog[]
  budget      Budget?

  @@index([clerkId])
}

// ■■ API KEY ■■■■■■■■■■■■■■■■■■■
```

```
  usageLogs   UsageLog[]
  budget      Budget?

  @@index([userId])
  @@index([slug])
}

// ■■ BUDGET ■■■
```

```
  id          String    @id @default(cuid())
  userId      String
  name        String
  url         String
  secret      String    // HMAC signing secret
  events      String[]  // ["budget.exceeded", "request.failed", ...]
  isActive    Boolean   @default(true)
  lastFiredAt DateTime?
  failCount   Int       @default(0)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  deliveries  WebhookDelivery[]

  @@index([userId])
}

// ■■ WEBHOOK DELIVERY ■■■
```

```
  });

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

1.4 — Redis Client

```
// src/lib/redis.ts
import { Redis } from 'ioredis';

const globalForRedis = global as unknown as { redis: Redis };

export const redis =
  globalForRedis.redis ??
  new Redis(process.env.REDIS_URL!, {
    maxRetriesPerRequest: 3,
    enableReadyCheck: false,
    retryStrategy: (times) => Math.min(times * 50, 2000),
  });

if (process.env.NODE_ENV !== 'production') {
  globalForRedis.redis = redis;
}

// Helpers utilitaires
export const cache = {
  get: (key: string) => redis.get(key),
  set: (key: string, val: string, ttlSeconds?: number) =>
    ttlSeconds ? redis.setex(key, ttlSeconds, val) : redis.set(key, val),
  del: (key: string) => redis.del(key),
  incr: (key: string) => redis.incr(key),
  incrby: (key: string, n: number) => redis.incrby(key, n),
};
```


© 2025 GateCtr

PHASE 2

Core API — SDK, Auth, Budget

Endpoints /complete · /chat · Budget Firewall · Auth middleware

2

05 Phase 2 — Core API

/complete · /chat · Budget Firewall · API Key auth

2.1 — Middleware d'authentification API

```
// src/lib/api-auth.ts
import { NextRequest } from 'next/server';
import { prisma } from './prisma';
import bcrypt from 'bcryptjs';

export async function authenticateApiKey(req: NextRequest) {
  const authHeader = req.headers.get('authorization');
  if (!authHeader?.startsWith('Bearer ')) {
    return { error: 'Missing or invalid Authorization header', status: 401 };
  }

  const rawKey = authHeader.replace('Bearer ', '');

  // API keys start with "gct_"
  if (!rawKey.startsWith('gct_')) {
    return { error: 'Invalid API key format', status: 401 };
  }

  const prefix = rawKey.substring(0, 12);

  // Fetch keys by prefix (faster than full scan)
  const keys = await prisma.apiKey.findMany({
    where: { prefix, isActive: true },
    include: { user: { include: { budget: true } } },
  });

  for (const key of keys) {
    const valid = await bcrypt.compare(rawKey, key.keyHash);
    if (valid) {
      // Update lastUsedAt async (non-blocking)
      prisma.apiKey.update({
        where: { id: key.id },
        data: { lastUsedAt: new Date() },
      }).catch(console.error);
      return { apiKey: key, user: key.user };
    }
  }

  return { error: 'Invalid API key', status: 401 };
}
```

2.2 — Endpoint POST /api/v1/complete

© 2025 GateCtr

```
// src/app/api/v1/complete/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { z } from 'zod';
import { authenticateApiKey } from '@/lib/api-auth';
import { checkBudget } from '@/lib/firewall';
import { optimizeContext } from '@/lib/optimizer';
import { routeModel } from '@/lib/router';
import { callLLM } from '@/lib/llm';
import { logUsage } from '@/lib/analytics';

const CompleteSchema = z.object({
  model:     z.string().default('auto'),
  messages:  z.array(z.object({
    role:    z.enum(['system', 'user', 'assistant']),
    content: z.string(),
  })),
  projectId: z.string().optional(),
  userId:    z.string().optional(),
  optimize:  z.boolean().default(true),
  stream:    z.boolean().default(false),
  maxTokens: z.number().optional(),
  temperature: z.number().min(0).max(2).optional(),
});

export async function POST(req: NextRequest) {
  // 1. Auth
  const auth = await authenticateApiKey(req);
  if (auth.error) {
    return NextResponse.json({ error: auth.error }, { status: auth.status });
  }
  const { user, apiKey } = auth;

  // 2. Parse body
  const body = await req.json();
  const parsed = CompleteSchema.safeParse(body);
  if (!parsed.success) {
    return NextResponse.json({ error: parsed.error.errors }, { status: 400 });
  }
  const input = parsed.data;

  // 3. Budget check
  const budgetCheck = await checkBudget(user.id, input.projectId);
  if (!budgetCheck.allowed) {
    return NextResponse.json({
      error: 'Budget limit exceeded',
      details: budgetCheck.reason,
      usage: budgetCheck.current,
    }, { status: 429 });
  }

  // 4. Optimize context
  const { messages, savedTokens } = input.optimize
    ? await optimizeContext(input.messages)
    : { messages: input.messages, savedTokens: 0 };

  // 5. Route model
  const { model, provider, llmKey } = await routeModel(
```


© 2025 GateCtr

```
    input.model,
    user,
    messages,
  );

  // 6. Call LLM
  const start = Date.now();
  const response = await callLLM({ model, provider, llmKey, messages,
    maxTokens: input.maxTokens, temperature: input.temperature });
  const latency = Date.now() - start;

  // 7. Log usage (async)
  logUsage({
    userId: user.id,
    projectId: input.projectId,
    apiKeyId: apiKey.id,
    model, provider,
    promptTokens: response.usage.prompt_tokens,
    completionTokens: response.usage.completion_tokens,
    savedTokens,
    costUsd: response.cost,
    latencyMs: latency,
    optimized: input.optimize,
    routed: input.model === 'auto',
  }).catch(console.error);

  return NextResponse.json({
    content: response.content,
    model,
    provider,
    usage: {
      prompt_tokens:      response.usage.prompt_tokens,
      completion_tokens:  response.usage.completion_tokens,
      total_tokens:       response.usage.total_tokens,
      saved_tokens:       savedTokens,
      cost_usd:           response.cost,
    },
    latency_ms: latency,
  });
}
```

2.3 — Budget Firewall

```
// src/lib/firewall.ts
import { prisma } from './prisma';
import { cache } from './redis';

export async function checkBudget(userId: string, projectId?: string) {
  const budget = await getBudget(userId, projectId);
  if (!budget) return { allowed: true };

  const today = new Date().toISOString().split('T')[0];
  const cacheKey = `budget:${userId}:${today}`;

  // Get daily usage from Redis cache (fast path)
```


© 2025 GateCtr

```
  const cached = await cache.get(cacheKey);
  const todayTokens = cached ? parseInt(cached) : await getDailyUsage(userId, today);

  if (budget.maxTokensPerDay && todayTokens >= budget.maxTokensPerDay) {
    if (budget.hardStop) {
      return {
        allowed: false,
        reason: `Daily token limit reached: ${todayTokens}/${budget.maxTokensPerDay}`,
        current: { tokens: todayTokens, limit: budget.maxTokensPerDay },
      };
    }
    // Soft limit: allow but trigger webhook
    triggerBudgetAlert(userId, 'budget.exceeded', { todayTokens, budget }).catch(console.error);
  }

  // Alert threshold (e.g. 80%)
  const pct = (todayTokens / (budget.maxTokensPerDay ?? Infinity)) * 100;
  if (pct >= budget.alertThreshold && pct < 100) {
    triggerBudgetAlert(userId, 'token.threshold', { percentage: pct, remaining:
budget.maxTokensPerDay! - todayTokens }).catch(console.error);
  }

  return { allowed: true, remaining: budget.maxTokensPerDay ? budget.maxTokensPerDay - todayTokens :
null };
}

async function getDailyUsage(userId: string, date: string): Promise<number> {
  const result = await prisma.usageLog.aggregate({
    where: { userId, createdAt: { gte: new Date(date), lt: new Date(date + 'T23:59:59') } },
    _sum: { totalTokens: true },
  });
  const tokens = result._sum.totalTokens ?? 0;
  // Cache for 60 seconds
  await cache.set(`budget:${userId}:${date}`, String(tokens), 60);
  return tokens;
}
```


© 2025 GateCtr

PHASE 3

Modules Métier

Context Optimizer · Model Router · Webhooks · Analytics

3

06 Phase 3 — Modules

Optimizer, Router, Webhook Engine, Analytics Queue

3.1 — Context Optimizer

```
// src/lib/optimizer.ts
import type { ChatMessage } from '@/types';

interface OptimizeResult {
  messages: ChatMessage[];
  savedTokens: number;
  compressionRatio: number;
}

export async function optimizeContext(
  messages: ChatMessage[],
  options = { maxContextTokens: 8000, keepLastN: 6 }
): Promise<OptimizeResult> {
  const original = estimateTokens(messages);

  // Strategy 1: Remove old messages (keep system + last N)
  let optimized = pruneHistory(messages, options.keepLastN);

  // Strategy 2: Compress system prompt (remove redundant whitespace)
  optimized = compressSystemPrompt(optimized);

  // Strategy 3: Truncate very long messages
  optimized = truncateLongMessages(optimized, 2000);

  const after = estimateTokens(optimized);
  const saved = original - after;

  return {
    messages: optimized,
    savedTokens: Math.max(0, saved),
    compressionRatio: after / original,
  };
}

function estimateTokens(messages: ChatMessage[]): number {
  // ~4 chars per token (rough estimate; use tiktoken for production)
  return messages.reduce((acc, m) => acc + Math.ceil(m.content.length / 4), 0);
}

function pruneHistory(messages: ChatMessage[], keepLast: number): ChatMessage[] {
  const system = messages.filter(m => m.role === 'system');
  const convo  = messages.filter(m => m.role !== 'system');
  const pruned = convo.slice(-keepLast);
```


© 2025 GateCtr

```
  return [...system, ...pruned];
}

function compressSystemPrompt(messages: ChatMessage[]): ChatMessage[] {
  return messages.map(m =>
    m.role === 'system'
      ? { ...m, content: m.content.replace(/\s+/g, ' ').trim() }
      : m
  );
}

function truncateLongMessages(messages: ChatMessage[], maxChars: number): ChatMessage[] {
  return messages.map(m =>
    m.content.length > maxChars
      ? { ...m, content: m.content.substring(0, maxChars) + '... [truncated]' }
      : m
  );
}
```

3.2 — Model Router

```
// src/lib/router.ts
import type { User, ChatMessage } from '@/types';

interface RouterResult {
  model:    string;
  provider: string;
  llmKey:   string;
}

// Scoring models: cost per 1K tokens (input/output avg in USD)
const MODEL_REGISTRY = [
  { id: 'gpt-4o-mini',     provider: 'openai',    costPer1k: 0.00015, quality: 0.82, contextWindow:
128000 },
  { id: 'gpt-4o',          provider: 'openai',    costPer1k: 0.005,   quality: 0.95, contextWindow:
128000 },
  { id: 'claude-haiku',    provider: 'anthropic', costPer1k: 0.00025, quality: 0.80, contextWindow:
200000 },
  { id: 'claude-sonnet',   provider: 'anthropic', costPer1k: 0.003,   quality: 0.92, contextWindow:
200000 },
  { id: 'mistral-small',   provider: 'mistral',   costPer1k: 0.0002,  quality: 0.78, contextWindow:
32000  },
  { id: 'mistral-large',   provider: 'mistral',   costPer1k: 0.002,   quality: 0.88, contextWindow:
32000  },
  { id: 'gemini-1.5-flash',provider: 'google',    costPer1k: 0.000075,quality: 0.79, contextWindow:
1000000},
];

export async function routeModel(
  requestedModel: string,
  user: User,
  messages: ChatMessage[],
  strategy: 'cost' | 'performance' | 'balanced' = 'balanced'
): Promise<RouterResult> {
  // Explicit model requested — bypass routing
```


© 2025 GateCtr

```
  if (requestedModel !== 'auto') {
    return resolveExplicitModel(requestedModel, user);
  }

  const estimatedTokens = messages.reduce((a, m) => a + Math.ceil(m.content.length / 4), 0);
  const plan = user.plan;

  // Filter eligible models based on plan
  const eligible = MODEL_REGISTRY.filter(m => {
    if (plan === 'FREE' && m.costPer1k > 0.001) return false;
    return m.contextWindow >= estimatedTokens;
  });

  // Score and rank
  const scored = eligible.map(m => ({
    ...m,
    score: strategy === 'cost'        ? -m.costPer1k
         : strategy === 'performance' ? m.quality
         : (m.quality * 0.6) - (m.costPer1k * 100 * 0.4),
  })).sort((a, b) => b.score - a.score);

  const selected = scored[0];

  return {
    model: selected.id,
    provider: selected.provider,
    llmKey: getUserLLMKey(user, selected.provider),
  };
}
```

3.3 — Webhook Dispatcher (BullMQ)

```
// src/lib/webhooks.ts
import { Queue } from 'bullmq';
import { redis } from './redis';

export const webhookQueue = new Queue('webhooks', {
  connection: redis,
  defaultJobOptions: {
    attempts: 3,
    backoff: { type: 'exponential', delay: 2000 },
    removeOnComplete: 100,
    removeOnFail: 500,
  },
});

export async function triggerWebhook(
  userId: string,
  event: string,
  payload: Record<string, unknown>
) {
  await webhookQueue.add('dispatch', {
    userId, event, payload,
    timestamp: new Date().toISOString(),
  });
```


© 2025 GateCtr

```
}

// workers/webhook.worker.ts
import { Worker } from 'bullmq';
import crypto from 'crypto';
import axios from 'axios';
import { prisma } from '../src/lib/prisma';
import { redis } from '../src/lib/redis';

new Worker('webhooks', async (job) => {
  const { userId, event, payload, timestamp } = job.data;

  const webhooks = await prisma.webhook.findMany({
    where: { userId, isActive: true, events: { has: event } },
  });

  for (const wh of webhooks) {
    const body = JSON.stringify({ event, timestamp, data: payload });
    const sig = crypto
      .createHmac('sha256', wh.secret)
      .update(body)
      .digest('hex');

    const start = Date.now();
    try {
      const res = await axios.post(wh.url, body, {
        headers: {
          'Content-Type': 'application/json',
          'X-GateCtr-Signature': `sha256=${sig}`,
          'X-GateCtr-Event': event,
        },
        timeout: 10000,
      });

      await prisma.webhookDelivery.create({
        data: { webhookId: wh.id, event, payload, status: res.status,
          responseMs: Date.now() - start, success: true },
      });
    } catch (err: any) {
      await prisma.webhookDelivery.create({
        data: { webhookId: wh.id, event, payload, status: err?.response?.status ?? 0,
          responseMs: Date.now() - start, success: false,
          error: err.message?.substring(0, 255) },
      });
    }
  }
}, { connection: redis, concurrency: 10 });
```


© 2025 GateCtr

PHASE 4

Frontend & Dashboard

Next.js App Router · shadcn/ui · Charts analytics · Dark mode

4

07 Phase 4 — Frontend & Dashboard

Layout, components, analytics UI

4.1 — shadcn/ui setup

```
# Initialiser shadcn/ui
pnpm dlx shadcn-ui@latest init

# Sélectionner : TypeScript, Default style, Slate color, CSS variables
# Cela crée : components.json, src/components/ui/, src/lib/utils.ts

# Ajouter les composants utilisés dans le projet
pnpm dlx shadcn-ui@latest add button card badge table dialog
pnpm dlx shadcn-ui@latest add dropdown-menu select input label
pnpm dlx shadctr-ui@latest add toast sheet tabs avatar progress
pnpm dlx shadcn-ui@latest add chart skeleton separator
```

4.2 — Layout Dashboard

```
// src/app/(dashboard)/layout.tsx
import { auth } from '@clerk/nextjs/server';
import { redirect } from 'next/navigation';
import { Sidebar } from '@/components/layout/Sidebar';
import { Header } from '@/components/layout/Header';
import { QueryProvider } from '@/components/providers/QueryProvider';

export default async function DashboardLayout({ children }) {
  const { userId } = auth();
  if (!userId) redirect('/sign-in');

  return (
    <QueryProvider>
      <div className="flex h-screen bg-background overflow-hidden">
        <Sidebar />
        <div className="flex flex-col flex-1 overflow-hidden">
          <Header />
          <main className="flex-1 overflow-y-auto p-6">
            {children}
          </main>
        </div>
      </div>
    </QueryProvider>
  );
}
```

4.3 — Dashboard Overview (Server Component)

© 2025 GateCtr

```
// src/app/(dashboard)/page.tsx
import { auth } from '@clerk/nextjs/server';
import { prisma } from '@/lib/prisma';
import { StatsCards } from '@/components/dashboard/StatsCards';
import { UsageChart } from '@/components/charts/UsageChart';
import { RecentLogs } from '@/components/dashboard/RecentLogs';

export default async function DashboardPage() {
  const { userId: clerkId } = auth();
  const user = await prisma.user.findUnique({
    where: { clerkId },
    include: { budget: true },
  });

  // Fetch last 7 days usage
  const sevenDaysAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);
  const usage = await prisma.usageLog.groupBy({
    by: ['createdAt'],
    where: { userId: user!.id, createdAt: { gte: sevenDaysAgo } },
    _sum: { totalTokens: true, savedTokens: true, costUsd: true },
    orderBy: { createdAt: 'asc' },
  });

  return (
    <div className="space-y-6">
      <h1 className="text-2xl font-bold">Dashboard</h1>
      <StatsCards usage={usage} budget={user?.budget} />
      <UsageChart data={usage} />
      <RecentLogs userId={user!.id} />
    </div>
  );
}
```

4.4 — Composant UsageChart

```
// src/components/charts/UsageChart.tsx
'use client';
import { AreaChart, Area, XAxis, YAxis, Tooltip, ResponsiveContainer } from 'recharts';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';

export function UsageChart({ data }) {
  const chartData = data.map(d => ({
    date: new Date(d.createdAt).toLocaleDateString('fr-FR', { weekday: 'short' }),
    tokens: d._sum.totalTokens ?? 0,
    saved:  d._sum.savedTokens  ?? 0,
    cost:   (d._sum.costUsd ?? 0).toFixed(4),
  }));

  return (
    <Card>
      <CardHeader>
        <CardTitle>Consommation tokens — 7 derniers jours</CardTitle>
      </CardHeader>
      <CardContent>
        <ResponsiveContainer width="100%" height={280}>
          <AreaChart data={chartData}>
```


© 2025 GateCtr

```
            <defs>
              <linearGradient id="tokensGrad" x1="0" y1="0" x2="0" y2="1">
                <stop offset="5%"  stopColor="#1B4F82" stopOpacity={0.3}/>
                <stop offset="95%" stopColor="#1B4F82" stopOpacity={0}/>
              </linearGradient>
              <linearGradient id="savedGrad" x1="0" y1="0" x2="0" y2="1">
                <stop offset="5%"  stopColor="#00B4C8" stopOpacity={0.3}/>
                <stop offset="95%" stopColor="#00B4C8" stopOpacity={0}/>
              </linearGradient>
            </defs>
            <XAxis dataKey="date" />
            <YAxis />
            <Tooltip formatter={(v, name) =>
              [v.toLocaleString(), name === 'tokens' ? 'Tokens utilisés' : 'Tokens économisés']
            }/>
            <Area type="monotone" dataKey="tokens" stroke="#1B4F82"
                  fill="url(#tokensGrad)" strokeWidth={2}/>
            <Area type="monotone" dataKey="saved"  stroke="#00B4C8"
                  fill="url(#savedGrad)"  strokeWidth={2}/>
          </AreaChart>
        </ResponsiveContainer>
      </CardContent>
    </Card>
  );
}
```


© 2025 GateCtr

08 Migrations & Base de Données

Prisma Migrate · Seeds · Backup · Neon

Workflow migrations Prisma

```
# ■■ DÉVELOPPEMENT ■■■
```

```
        },
      },
    },
  });

  // Créer un projet de test
  const project = await prisma.project.upsert({
    where: { slug: 'test-project' },
    update: {},
    create: {
      userId: user.id,
      name: 'Test Project',
      slug: 'test-project',
    },
  });

  // Créer un webhook de test
  await prisma.webhook.create({
    data: {
      userId: user.id,
      name: 'Dev Slack',
      url: 'https://hooks.slack.com/test',
      secret: 'test-secret-key',
      events: ['budget.exceeded', 'request.failed'],
    },
  });

  console.log('■ Seed complete:', { user: user.email, project: project.slug });
}

main().catch(console.error).finally(() => prisma.$disconnect());
```

```
# Ajouter dans package.json
# "prisma": { "seed": "tsx prisma/seed.ts" }
npx prisma db seed
```

Backup PostgreSQL

```
# Backup manuel depuis Docker
docker exec gatectr_postgres pg_dump \
  -U gatectr -d gatectr_dev \
  --format=custom \
  --compress=9 \
  > backups/gatectr_$(date +%Y%m%d_%H%M%S).dump

# Restaurer
docker exec -i gatectr_postgres pg_restore \
  -U gatectr -d gatectr_dev \
  --clean --if-exists \
  < backups/gatectr_20250115_120000.dump

# Script de backup automatique (cron)
#!/bin/bash
BACKUP_DIR=/backups
DATE=$(date +%Y%m%d_%H%M%S)
```


© 2025 GateCtr

```
docker exec gatectr_postgres pg_dump -U gatectr -d gatectr_dev \
  --format=custom > "$BACKUP_DIR/gatectr_$DATE.dump"
# Garder 30 jours
find $BACKUP_DIR -name "*.dump" -mtime +30 -delete
```

Configuration Neon (PostgreSQL serverless)

```
# Pour Vercel + Neon (production serverless)
pnpm add @neondatabase/serverless

# src/lib/prisma.ts (version Neon pooled)
import { PrismaClient } from '@prisma/client';
import { Pool, neonConfig } from '@neondatabase/serverless';
import { PrismaNeon } from '@prisma/adapter-neon';
import ws from 'ws';

neonConfig.webSocketConstructor = ws;
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
const adapter = new PrismaNeon(pool);

export const prisma = new PrismaClient({ adapter });

# Dans prisma/schema.prisma, ajouter :
# generator client {
#   provider = "prisma-client-js"
#   previewFeatures = ["driverAdapters"]
# }
```


© 2025 GateCtr

09 Tests : Unit · Integration · E2E

Vitest · Testing Library · Playwright

Configuration Vitest

```
// vitest.config.ts
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./tests/setup.ts'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'json', 'html'],
      thresholds: { lines: 80, functions: 80, branches: 75 },
    },
  },
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
});
```

```
// tests/setup.ts
import '@testing-library/jest-dom';
import { vi } from 'vitest';

// Mock Prisma globally
vi.mock('@/lib/prisma', () => ({
  prisma: {
    user:     { findUnique: vi.fn(), create: vi.fn(), update: vi.fn() },
    apiKey:   { findMany: vi.fn(), update: vi.fn() },
    usageLog: { create: vi.fn(), aggregate: vi.fn(), groupBy: vi.fn() },
    budget:   { findFirst: vi.fn() },
    webhook:  { findMany: vi.fn() },
  },
}));

// Mock Redis
vi.mock('@/lib/redis', () => ({
  redis: { get: vi.fn(), set: vi.fn(), setex: vi.fn(), del: vi.fn() },
  cache: { get: vi.fn(), set: vi.fn(), del: vi.fn(), incr: vi.fn() },
}));
```

Tests unitaires — Budget Firewall

```
// tests/unit/firewall.test.ts
import { describe, it, expect, vi, beforeEach } from 'vitest';
import { checkBudget } from '@/lib/firewall';
```


© 2025 GateCtr

```
import { prisma } from '@/lib/prisma';
import { cache } from '@/lib/redis';

describe('Budget Firewall', () => {
  beforeEach(() => { vi.clearAllMocks(); });

  it('should allow request when no budget configured', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue(null);
    const result = await checkBudget('user_123');
    expect(result.allowed).toBe(true);
  });

  it('should block when hard stop and limit exceeded', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue({
      maxTokensPerDay: 1000, hardStop: true, alertThreshold: 80,
    } as any);
    vi.mocked(cache.get).mockResolvedValue('1001');  // Over limit

    const result = await checkBudget('user_123');
    expect(result.allowed).toBe(false);
    expect(result.reason).toContain('Daily token limit');
  });

  it('should allow but warn at threshold (soft limit)', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue({
      maxTokensPerDay: 1000, hardStop: false, alertThreshold: 80,
    } as any);
    vi.mocked(cache.get).mockResolvedValue('820');  // 82% — over threshold

    const result = await checkBudget('user_123');
    expect(result.allowed).toBe(true);
  });

  it('should return remaining tokens', async () => {
    vi.mocked(prisma.budget.findFirst).mockResolvedValue({
      maxTokensPerDay: 10000, hardStop: false, alertThreshold: 80,
    } as any);
    vi.mocked(cache.get).mockResolvedValue('3000');

    const result = await checkBudget('user_123');
    expect(result.allowed).toBe(true);
    expect(result.remaining).toBe(7000);
  });
});
```

Tests unitaires — Context Optimizer

```
// tests/unit/optimizer.test.ts
import { describe, it, expect } from 'vitest';
import { optimizeContext } from '@/lib/optimizer';

describe('Context Optimizer', () => {
  it('should prune old messages keeping system + last N', async () => {
    const messages = [
      { role: 'system',    content: 'You are helpful.' },
```


© 2025 GateCtr

```
      { role: 'user',      content: 'Message 1' },
      { role: 'assistant', content: 'Response 1' },
      { role: 'user',      content: 'Message 2' },
      { role: 'assistant', content: 'Response 2' },
      { role: 'user',      content: 'Message 3' },
    ];
    const { messages: out } = await optimizeContext(messages, { keepLastN: 4, maxContextTokens: 8000
});
    expect(out[0].role).toBe('system');
    expect(out.filter(m => m.role !== 'system').length).toBeLessThanOrEqual(4);
  });

  it('should compress whitespace in system prompt', async () => {
    const messages = [{ role: 'system', content: 'You   are   helpful.' }];
    const { messages: out } = await optimizeContext(messages);
    expect(out[0].content).toBe('You are helpful.');
  });

  it('should report saved tokens', async () => {
    const long = 'word '.repeat(500);
    const messages = [
      { role: 'system', content: '  System  prompt  with  spaces  '.repeat(10) },
      { role: 'user',   content: long },
    ];
    const result = await optimizeContext(messages);
    expect(result.savedTokens).toBeGreaterThan(0);
  });
});
```

Tests E2E — Playwright

```
// tests/e2e/dashboard.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Dashboard', () => {
  test.beforeEach(async ({ page }) => {
    // Simuler un utilisateur connecté (via session cookie)
    await page.goto('/');
    // Utiliser Clerk test helper ou mock
  });

  test('should display usage stats', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page.getByText('Dashboard')).toBeVisible();
    await expect(page.locator('[data-testid="stats-cards"]')).toBeVisible();
  });

  test('should navigate to analytics', async ({ page }) => {
    await page.goto('/dashboard');
    await page.click('text=Analytics');
    await expect(page).toHaveURL('/dashboard/analytics');
  });

  test('should create a webhook', async ({ page }) => {
    await page.goto('/dashboard/webhooks');
```


© 2025 GateCtr

```
    await page.click('[data-testid="add-webhook-btn"]');
    await page.fill('[name="webhook-name"]', 'Test Webhook');
    await page.fill('[name="webhook-url"]', 'https://hooks.example.com/test');
    await page.click('[data-testid="save-webhook"]');
    await expect(page.getByText('Test Webhook')).toBeVisible();
  });
});

# playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests/e2e',
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  webServer: {
    command: 'pnpm dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Commandes de test

```
# Unit + Integration tests
pnpm test                    # Run all tests
pnpm test:watch              # Watch mode
pnpm test:coverage           # With coverage report

# E2E tests
pnpm playwright install      # Install browsers (first time)
pnpm test:e2e                # Run Playwright
pnpm test:e2e --ui           # Playwright UI mode
pnpm test:e2e --headed       # Visible browser

# Check coverage thresholds
pnpm test:coverage --reporter=html
# Open coverage/index.html
```

Checklist des tests par phase

- ■ Unit: Budget Firewall — all budget scenarios (hard stop, soft limit, threshold)
- ■ Unit: Context Optimizer — pruning, compression, token estimation
- ■ Unit: Model Router — scoring, eligible models, explicit model override
- ■ Unit: API Key Auth — valid key, expired key, invalid prefix, inactive key

© 2025 GateCtr

- Unit: Webhook Dispatcher — HMAC signature, retry logic, delivery logging
- Integration: POST /api/v1/complete — full request cycle with mocked LLM
- Integration: Budget Firewall + Redis — real Redis counter, threshold triggers
- Integration: Prisma — CRUD users, projects, usage logs (test DB)
- E2E: Dashboard login → analytics view → webhook creation
- E2E: API key creation, copy, usage
- E2E: Budget settings update → alert trigger
- Performance: /api/v1/complete < 200ms (ex LLM latency)

© 2025 GateCtr

PHASE 5

CI/CD & DevOps

GitHub Actions · Vercel · Monitoring · Alerting · Security

5

10 CI/CD & DevOps Complet

GitHub Actions · Vercel Deploy · Sentry · Axiom · Security

10.1 — GitHub Actions : CI Pipeline

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  # ■■ Lint & Type Check ■■■
```

```
  name: Integration Tests
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: gatectr
        POSTGRES_PASSWORD: secret
        POSTGRES_DB: gatectr_test
      ports: ['5432:5432']
      options: >-
        --health-cmd pg_isready
        --health-interval 10s --health-timeout 5s --health-retries 5
    redis:
      image: redis:7-alpine
      ports: ['6379:6379']
      options: >-
        --health-cmd "redis-cli ping"
        --health-interval 10s --health-timeout 5s --health-retries 5
  env:
    DATABASE_URL: postgresql://gatectr:secret@localhost:5432/gatectr_test
    REDIS_URL: redis://localhost:6379
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
      with: { version: 8 }
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'pnpm' }
    - run: pnpm install --frozen-lockfile
    - run: npx prisma migrate deploy
    - run: npx prisma db seed
    - run: pnpm test:integration

# ■■ E2E Tests ■■■
```

10.2 — GitHub Actions : Deploy Pipeline

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-production:
    name: Deploy to Vercel (Production)
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with: { version: 8 }
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile

      # Run migrations before deploy
      - name: Run DB Migrations
        run: npx prisma migrate deploy
        env:
          DATABASE_URL: ${{ secrets.PRODUCTION_DATABASE_URL }}

      # Deploy to Vercel
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          vercel-args: '--prod'

      # Notify Slack on success
      - name: Notify Slack
        if: success()
        uses: slackapi/slack-github-action@v1
        with:
          payload: |
            {"text": "■ GateCtr deployed to production — ${{ github.sha }}"}
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}

      # Notify on failure
      - name: Notify Failure
        if: failure()
        uses: slackapi/slack-github-action@v1
        with:
          payload: |
            {"text": "■ GateCtr deploy FAILED — ${{ github.sha }} — check GitHub Actions"}
        env:
          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_DEPLOY_WEBHOOK }}
```


© 2025 GateCtr

10.3 — Vercel Configuration

```
// vercel.json
{
  "framework": "nextjs",
  "buildCommand": "pnpm build",
  "installCommand": "pnpm install --frozen-lockfile",
  "outputDirectory": ".next",
  "regions": ["cdg1", "iad1"],
  "env": {
    "NEXT_TELEMETRY_DISABLED": "1"
  },
  "headers": [
    {
      "source": "/api/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options",    "value": "nosniff"    },
        { "key": "X-Frame-Options",           "value": "DENY"       },
        { "key": "X-XSS-Protection",          "value": "1; mode=block" },
        { "key": "Strict-Transport-Security", "value": "max-age=31536000; includeSubDomains" },
        { "key": "Cache-Control",             "value": "no-store"   }
      ]
    }
  ],
  "rewrites": [
    { "source": "/sdk/:path*", "destination": "/api/v1/:path*" }
  ]
}
```

10.4 — Variables Vercel (secrets CI/CD)

Variable	Env	Source
`DATABASE_URL`	Production	Neon / Supabase dashboard
`REDIS_URL`	Production	Upstash dashboard
`CLERK_SECRET_KEY`	All	Clerk dashboard → API Keys
`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	All	Clerk dashboard → API Keys
`CLERK_WEBHOOK_SECRET`	All	Clerk dashboard → Webhooks
`ENCRYPTION_KEY`	Production	Générer: openssl rand -hex 32
`WEBHOOK_SIGNING_SECRET`	Production	Générer: openssl rand -hex 32
`SENTRY_DSN`	All	Sentry project settings
`SENTRY_AUTH_TOKEN`	CI/CD	Sentry → Account → API Tokens
`VERCEL_TOKEN`	CI/CD	Vercel → Account Settings → Tokens
`VERCEL_ORG_ID`	CI/CD	Vercel → Team Settings
`VERCEL_PROJECT_ID`	CI/CD	Vercel → Project Settings

© 2025 GateCtr

`SLACK_DEPLOY_WEBHOOK`	CI/CD	Slack → Incoming Webhooks

10.5 — Sentry Error Tracking

```
# Installation
pnpm add @sentry/nextjs

# Configuration automatique
npx @sentry/wizard@latest -i nextjs

# sentry.client.config.ts
import * as Sentry from '@sentry/nextjs';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,
  replaysOnErrorSampleRate: 1.0,
  replaysSessionSampleRate: 0.05,
  integrations: [
    Sentry.replayIntegration({
      maskAllText: true,
      blockAllMedia: true,
    }),
  ],
});

# Capturer une erreur métier
try {
  await callLLM(params);
} catch (error) {
  Sentry.captureException(error, {
    extra: { model, provider, userId },
    tags: { phase: 'llm_call' },
  });
  throw error;
}
```

10.6 — Monitoring & Alerting Stack

Outil	Usage	Intégration
Sentry	Error tracking, stack traces, perf	SDK @sentry/nextjs
Axiom	Log aggregation, recherche	SDK @axiomhq/next
Vercel Analytics	Web vitals, perf pages	Natif Vercel
Uptime Robot	Ping /api/health toutes les 5min	URL monitor gratuit
Grafana Cloud	Dashboards infra Redis/PG	Agent Grafana
GitHub Actions	Build / test status	Natif GitHub
Slack	Alertes deploy, erreurs critiques	Webhook Incoming

© 2025 GateCtr

PagerDuty	On-call alerting enterprise	Sentry integration

10.7 — Health Check Endpoint

```
// src/app/api/health/route.ts
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';
import { redis } from '@/lib/redis';

export async function GET() {
  const checks = {
    status: 'ok',
    timestamp: new Date().toISOString(),
    version: process.env.npm_package_version ?? '1.0.0',
    services: {
      database: 'unknown',
      redis:    'unknown',
    },
  };

  try {
    await prisma.$queryRaw`SELECT 1`;
    checks.services.database = 'healthy';
  } catch {
    checks.services.database = 'unhealthy';
    checks.status = 'degraded';
  }

  try {
    await redis.ping();
    checks.services.redis = 'healthy';
  } catch {
    checks.services.redis = 'unhealthy';
    checks.status = 'degraded';
  }

  const statusCode = checks.status === 'ok' ? 200 : 503;
  return NextResponse.json(checks, { status: statusCode });
}
```

10.8 — Sécurité & Hardening

Mesure	Implémentation	Priorité
HTTPS forcé	Vercel / HSTS header	■ Critique
Auth API Key	bcrypt hash + prefix lookup	■ Critique
HMAC Webhook Signing	SHA-256 sur chaque payload	■ Critique
Rate Limiting	Redis sliding window par IP/user	■ Critique
Input Validation	Zod sur tous les endpoints	■ Critique
Secrets encryption	AES-256 pour clés LLM stockées	■ Critique

© 2025 GateCtr

SQL Injection	Prisma parameterized queries (natif)	■ Élevé
XSS Protection	CSP headers, DOMPurify si user content	■ Élevé
CORS configuré	Whitelist origins dans Next.js	■ Élevé
Audit logs	UsageLog + WebhookDelivery immutables	■ Élevé
Dependency scanning	npm audit / Snyk dans CI	■ Moyen
Secrets scan	git-secrets + GitHub secret scanning	■ Moyen
Container scanning	Docker Scout dans CI pipeline	■ Moyen

10.9 — Rate Limiting (Redis)

```
// src/lib/rate-limit.ts
import { cache } from './redis';

interface RateLimitResult {
  allowed: boolean;
  remaining: number;
  reset: number;
}

export async function rateLimit(
  key: string,
  limit: number,
  windowSeconds: number
): Promise<RateLimitResult> {
  const now = Math.floor(Date.now() / 1000);
  const windowStart = now - windowSeconds;
  const redisKey = `ratelimit:${key}:${Math.floor(now / windowSeconds)}`;

  const current = await cache.incr(redisKey);
  if (current === 1) {
    // Set expiry on first request in window
    await cache.set(redisKey, String(current), windowSeconds + 1);
  }

  return {
    allowed: current <= limit,
    remaining: Math.max(0, limit - current),
    reset: Math.ceil(now / windowSeconds) * windowSeconds,
  };
}

// Usage in API route
const { allowed, remaining } = await rateLimit(
  `api:${userId}`,
  100,   // 100 requests
  60     // per 60 seconds
);

if (!allowed) {
  return NextResponse.json({ error: 'Rate limit exceeded' }, {
```


© 2025 GateCtr

```
    status: 429,
    headers: { 'X-RateLimit-Remaining': '0' },
  });
}
```

10.10 — Checklist DevOps finale

- Docker Compose dev stack : app + postgres + redis + worker
- Dockerfile multi-stage production (node:20-alpine)
- GitHub Actions CI : lint + type-check + unit tests + integration
- GitHub Actions Deploy : migrate + Vercel production + Slack notify
- Vercel project configuré avec regions EU + headers sécurité
- Variables d'environnement dans Vercel (prod) + Doppler (sync)
- Prisma migrations versionnées et appliquées via CI
- Sentry configuré (client + server + edge)
- Health check endpoint /api/health (DB + Redis)
- Rate limiting Redis sur tous les endpoints API publics
- Uptime Robot : ping /api/health toutes les 5 minutes
- Backup PostgreSQL automatique (daily, 30j retention)
- Docker Scout scan dans CI (security vulnerabilities)
- Axiom log aggregation configuré
- PagerDuty on-call routing pour incidents critiques

© 2025 GateCtr

gatectr.io · Developer Documentation v1.0 · Confidential
© 2025 GateCtr — All rights reserved